Chapter 1

Sorting

참고문헌:

1. R.Sedgewick and K. Wayne, Algorithms (4th Ed.), Addison-Wesley.
2. E. Horowitz, S. Sahni, S. Anderson-Freed, Fundamentals of Data Structures in C, Silicon Press, 2nd Edition.

강의 내용

- 소개
- 기본적인 정렬 알고리즘
- 선형 정렬 알고리즘
- 병합 정렬(Merge Sort)
- 외부 정렬(External Sort)

1. 소개

- 정렬 알고리즘 (Sorting Algorithm)
  - 리스트의 항목들을 특정한 순서에 따라 재배치
    - 검색, 비교 등의 연산이 용이
  - 초창기에는 전체 계산 처리량의 약 30%

- 정렬 알고리즘의 분류
  - 내부 정렬/외부 정렬
    - 데이터 수와 메모리 용량간의 관계
  - 비교 기반 정렬/선형 정렬
    - 데이터 값에 대한 제한 여부

2. 기본적인 정렬 알고리즘

- Java를 이용한 정렬 알고리즘 구현
- 선택 정렬 (Selection Sort)
- 삽입 정렬 (Insertion Sort)
- 쉘 정렬 (Shell Sort)

2.1 Java를 이용한 정렬 알고리즘 구현

- 알고리즘의 구성
  - 각 정렬 알고리즘에 대해 별도의 클래스 생성
    - 예:Selection, Insertion, Shell, …
  - static method로 sort() 정의
    - 입력: Comparable의 배열
  - 추가로 less(), exch(), show(), isSorted() 등의 static method 정의

- Comparable 인터페이스
  - 구현하는 클래스의 객체들간에 natural order 정의
    - Collections.sort() 등의 알고리즘에서 사용
  - int compareTo() 를 구현하여야 함
    - a.compareTo(b): 음수(a<b), 0 (a==b), 양수(a>b)

AbstractSort 클래스

```
public abstract class AbstractSort {
    public static void sort(Comparable[] a) { };

    protected static boolean less(Comparable v, Comparable w)
    {   return v.compareTo(w) < 0;   }

    protected static void exch(Comparable[] a, int i, int j)
    {   Comparable t = a[i]; a[i] = a[j];a[j] = t;   }

    protected static void show(Comparable[] a) {
        for (int i = 0; i < a.length; i++)  System.out.print(a[i] + " ");
        System.out.println();
    }

    protected static boolean isSorted(Comparable[] a) {
        for (int i = 1; i < a.length; i++)
            if (less(a[i], a[i-1]))   return false;
        return true;
}   }
```

Comparable 인터페이스의 예

```
public class Date implements Comparable<Date> {
    private final int month;
    private final int day;
    private final int year;
    public Date(int month, int day, int year) {
        if (!isValid(month, day, year)) throw new IllegalArgumentException("Invalid date");
        this.month = month;
        this.day   = day;
    }
    public int compareTo(Date that) {
        if (this.year  < that.year)  return -1;
        if (this.year  > that.year)  return +1;
        if (this.month < that.month) return -1;
        if (this.month > that.month) return +1;
        if (this.day < that.day)   return -1;
        if (this.day > that.day)   return +1;
        return 0;
    }
    …
```

2.2 선택 정렬 (Selection Sort)

- 알고리즘
  - 리스트에서 최소값을 찾는다.
  - 이 값을 현재 위치의 값과 교환한다.
  - 현재 위치를 다음으로 이동하면서 반복한다.

- 성능
  - 비교 연산: ~ $N^2/2$, 교환 연산: ~ N

- 특징
  - 실행 시간이 입력 데이터의 배치와 무관
  - 데이터 이동이 최소화
  - 제자리 정렬 (부가적인 저장 공간이 필요 없다.)

구현

```
public class Selection extends AbstractSort {
  public static void sort(Comparable[] a) {
    int N = a.length;
    for (int i = 0; i < N - 1; i++) {
      int min = i;
      for (int j = i+1; j < N; j++) {
        if (less(a[j], a[min]))
          min = j;
      }
      exch(a, i, min);
    }
    assert isSorted(a);
  };

  public static void main(String[] args) {
    Integer[] a = {10, 4, 5, 2, 1, 8, 3, 6};
    Selection.sort(a);
    Selection.show(a);
  }
}
```

2.3 삽입 정렬 (Insertion Sort)

- 알고리즘
  - 현재 위치를 i라고 하자. (0 < i < n)
  - i번째 원소를 0부터 i-1까지 정렬된 리스트에 추가
  - i를 n-1까지 증가하면서 반복한다.

- 성능
  - 최악 – 비교 연산: ~ $N^2/2$, 교환 연산: ~ $N^2/2$
  - 최선 – 비교 연산: ~ N, 교환 연산: 0

- 특징
  - 실행 시간이 입력 데이터의 초기 배치에 따라 결정
  - 데이터 크기가 클 경우?

구현

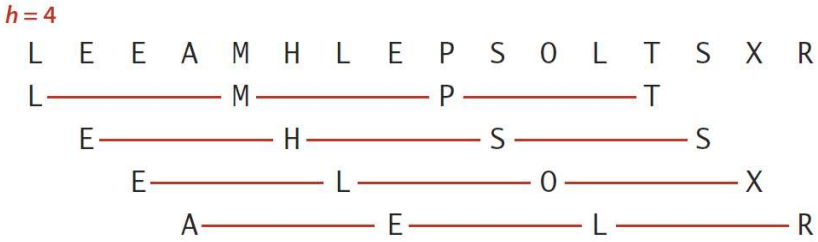
```
public class Insertion extends AbstractSort {
  public static void sort(Comparable[] a) {
    int N = a.length;
    for (int i = 1; i < N; i++) {
      for (int j = i; j > 0 && less(a[j], a[j-1]); j--) {
        exch(a, j, j-1);
      }
    }
    assert isSorted(a);
  }

  public static void main(String[] args) {
    Integer[] a = {10, 4, 5, 2, 1, 8, 3, 6};
    Insertion.sort(a);
    Insertion.show(a);
  }
}
```

2.4 쉘 정렬 (Shell Sort)

- 삽입 정렬의 문제점
  - 바로 앞의 원소와 비교
  - 가장 작은 원소가 마지막에 저장될 경우, n-1번의 교환 연산이 발생
    ← 정렬 시간이 길어지는 원인

- 쉘 정렬의 기본 개념
  - h개 만큼 떨어진 원소들과 비교 후 자리 바꿈
    - 한번의 교환 연산으로 h만큼 이동 가능
  - h-sorted array
    - h번째 항목들은 정렬된 배열



쉘 정렬 알고리즘

- 알고리즘
  - $h_r, h_{r-1}, \ldots h_1$까지의 수열을 가정 ($h_1 = 1, h_{i-1} < h_i$)
  - 처음에는 h_r개 떨어진 원소들간에 삽입 정렬
  - 이후 h_{r-1}부터 h_1까지 h를 줄이면서 삽입 정렬
    - h_1의 경우, 일반적인 삽입 정렬

- 특징
  - h의 값이 클 경우 (초기 단계), 리스트의 길이가 짧다.
  - h의 값이 작을 경우 (끝 단계), 리스트는 부분적으로 정렬된 상태

- h-수열의 할당 방법
  - $h_1 = 1$이므로 어떤 값을 할당하더라도 마지막 단계에서 정렬은 됨
  - 수열이 길면 단계 수 증가, 수열이 짧으면 성능 향상 정도가 미약
  - 최적의 수열은 아직 발견되지 않고 있으며,
  - 일반적으로 $h_i = 3 * h_{i-1} + 1$의 수열을 많이 사용: 복잡도 = $O(N^{1.5})$

알고리즘의 예

input	S	H	E	L	L	S	O	R	T	E	X	A	M	P	L	E
13-sort	P	H	E	L	L	S	O	R	T	E	X	A	M	S	L	E
	P	H	E	L	L	S	O	R	T	E	X	A	M	S	L	E
	P	H	E	L	L	S	O	R	T	E	X	A	M	S	L	E
4-sort	L	H	E	L	P	S	O	R	T	E	X	A	M	S	L	E
	L	H	E	L	P	S	O	R	T	E	X	A	M	S	L	E
	L	H	E	L	P	S	O	R	T	E	X	A	M	S	L	E
	L	H	E	L	P	S	O	R	T	E	X	A	M	S	L	E
	L	H	E	L	P	S	O	R	T	E	X	A	M	S	L	E
	L	E	E	L	P	H	O	R	T	S	X	A	M	S	L	E
	L	E	E	L	P	H	O	R	T	S	X	A	M	S	L	E
	L	E	E	A	P	H	O	L	T	S	X	R	M	S	L	E
	L	E	E	A	M	H	O	L	P	S	X	R	T	S	L	E
	L	E	E	A	M	H	O	L	P	S	X	R	T	S	L	E
	L	E	E	A	M	H	L	L	P	S	O	R	T	S	X	E
	L	E	E	A	M	H	L	E	P	S	O	L	T	S	X	R
1-sort	E	L	E	A	M	H	L	E	P	S	O	L	T	S	X	R
	E	E	L	A	M	H	L	E	P	S	O	L	T	S	X	R
	A	E	E	L	M	H	L	E	P	S	O	L	T	S	X	R
	A	E	E	L	M	H	L	E	P	S	O	L	T	S	X	R
	A	E	E	H	L	M	L	E	P	S	O	L	T	S	X	R
	A	E	E	H	L	L	M	E	P	S	O	L	T	S	X	R
	A	E	E	E	H	L	L	M	P	S	O	L	T	S	X	R
	A	E	E	E	H	L	L	M	P	S	O	L	T	S	X	R
	A	E	E	E	H	L	L	M	P	S	O	L	T	S	X	R
	A	E	E	E	H	L	L	M	O	P	S	L	T	S	X	R
	A	E	E	E	H	L	L	L	M	O	P	S	T	S	X	R
	A	E	E	E	H	L	L	L	M	O	P	S	T	S	X	R
	A	E	E	E	H	L	L	L	M	O	P	S	S	T	X	R
	A	E	E	E	H	L	L	L	M	O	P	S	S	T	X	R
	A	E	E	E	H	L	L	L	M	O	P	R	S	S	T	X
result	A	E	E	E	H	L	L	L	M	O	P	R	S	S	T	X

구현

```
public class Shell extends AbstractSort {
    public static void sort(Comparable[] a) {
        int N = a.length;
        int h = 1;
        while (h < N/3) h = 3*h + 1;        // h sequence:  1, 4, 13, 40, 121, 364, 1093, ...

        while (h >= 1) {    // h-sort the array
            for (int i = h; i < N; i++)
                for (int j = i; j >= h && less(a[j], a[j-h]); j -= h)
                    exch(a, j, j-h);        // h만큼 떨어진 원소들과 삽입 정렬
            h /= 3;
        }
    }

    public static void main(String[] args) {
        Integer[] a = {10, 4, 5, 2, 1, 8, 3, 6};
        Shell.sort(a);
        Shell.show(a);
}   }
```

3. 선형 정렬 알고리즘

- 키의 비교에 기반한 알고리즘들
  - 키에 대한 사전 정보를 가정하지 않고, 키의 비교에 의해서만 정렬하는 알고리즘
  - 선택 정렬, 삽입 정렬, 쉘 정렬, …
  - 복잡도: $O(n \log_2 n)$

- 선형 정렬 알고리즘
  - 키에 대한 추가적인 제약 조건(정보)을 가정
    - 예: 키 값의 범위, 키의 기수(자리 수) 등
  - 계수 정렬 (Counting Sort)
  - 버킷 정렬 (Bucket Sort)
  - 기수 정렬 (Radix Sort)

참고문헌:

1. 나라심하 카루만치, 다양한 예제로 학습하는 데이터 구조와 알고리즘 for Java, 인사이트.
2. 자료구조 책 (7.7절)

3.1 계수 정렬 (Counting Sort)

- 키 값이 0 ~ K-1 사이의 정수일 경우 적용 가능
  - 입력 배열: A[n]
  - 임시 배열: C[K]
  - 결과 배열: B[n]

- 기본 개념
  - A에 나오는 모든 값들에 대해 그 빈도수를 C에 계산
  - A[i] = 72이며, C[0] 부터 C[71]까지의 합이 53일 때, B[53]에 72를 저장

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
A	10	4	0	5	8	0	8	3	6	1	0
C	3	1	0	1	1	1	1	0	2	0	1
C'	3	4	4	5	6	7	8	8	10	10	11
B											

구현

```
public class Counting {
    public static int[] sort(int[] A, int K) {
        int i, N = A.length;
        int[] C = new int[K], B = new int[N];

        for (i = 0; i < N; i++)   C[A[i]]++;          // 원소의 빈도수를 합산
        for (i = 1; i < K; i++)   C[i] += C[i-1];     // i 값이 저장될 위치를 계산
        for (i = N-1; i >= 0; i--)                    // 왜 뒤에서부터 저장할까?
            B[--C[A[i]]] = A[i];                      // 정렬된 배열을 생성
        return B;
    }

public static void main(String[] args) {
    int[] A = {10, 4, 5, 8, 1, 8, 3, 6}, B;
    B = Counting.sort(A, 11);
    for (int i = 0; i < B.length; i++)
        System.out.print(B[i] + " ");
    System.out.println();
}  }
```

시간 복잡도: O(N + K)
공간 복잡도: O(N + K)

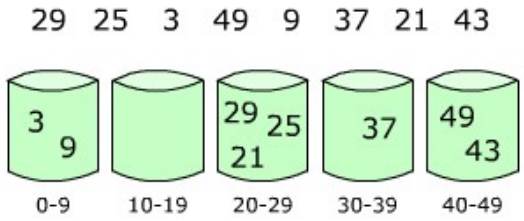
K의 크기가 N과 비슷하다면, O(N)
복잡도로 정렬 가능!!!

But, K = N^3이라면, 복잡도 = O(N^3)

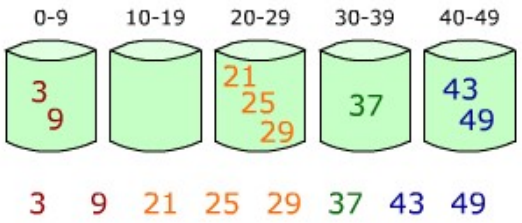
3.2 버킷 정렬 (Bucket Sort)

- 계수 정렬에서 여러 개의 구간을 하나의 버킷으로 통합
  - 버킷의 크기 = 1일 경우 → 계수 정렬
  - 각 버킷을 정렬하는 별도 단계를 갖는다. (기수 정렬)

- 알고리즘의 동작 과정
  - 초기에 비어 있는 버킷을 준비
  - 분할 단계: 입력 배열의 원소들을 해당되는 버킷에 저장
  - 정렬 단계: 각 버킷을 정렬
  - 수집 단계: 버킷을 차례로 방문하여 원소들을 원 배열에 저장



분할 단계



정렬 및 수집 단계

- 버킷 수가 2인 정렬 알고리즘?

3.3 기수 정렬 (Radix Sort)

- 키가 d개의 요소들로 구성된다고 가정
  - 예 1: 모든 키들이 세 자리 숫자
  - 예 2: 트럼프 카드 (무늬와 숫자의 두 가지 요소)

- 정렬 규칙
  - 높은 자리 요소가 우선
  - 높은 자리가 동일할 경우, 낮은 자리를 비교
  - 예: 트럼프 카드
    - K1 (무늬: 높은 자리): ♣ < ♥ < ♦ < ♠
    - K2 (숫자: 낮은 자리): 2 < 3 < … < J < Q < K < A
    - 정렬 순서: 2♣, … , A♣, …, 2♠, … , A♠

두 가지 정렬 방법

- 높은 자리 우선 정렬 (MSD sort)
  - K1 기준으로 버킷 정렬
  - 각 버킷에 대해 K2 기준으로 다시 버킷 정렬
  - 예: 트럼프 카드
    - 첫번째 단계: 무늬별로 4개의 버킷에 분할
    - 두번째 단계: 각 버킷에서 숫자별로 다시 정렬
    - 마지막 단계: 무늬 버킷을 순서대로 합친다

- 낮은 자리 우선 정렬 (LSD sort)
  - K2 기준으로 버킷 정렬
  - 각 버킷들을 합친다.
  - 다시 K1 기준으로 버킷 정렬 (이때 stable sorting 알고리즘 사용)
  - 버킷별로 별도 정렬 단계가 필요 없으므로 MSD보다 많이 사용

LSD를 이용한 기수 정렬의 예 (1)

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
179	208	306	93	859	984	55	9	271	33

C[0]	C[1]	C[2]	C[3]	C[4]	C[5]	C[6]	C[7]	C[8]	C[9]
	1		2	1	1	1		1	3
0	1	1	3	4	5	6	6	7	10

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
271	93	33	984	55	306	208	179	859	9

C[0]	C[1]	C[2]	C[3]	C[4]	C[5]	C[6]	C[7]	C[8]	C[9]
3			1		2		2	1	1
3	3	3	4	4	6	6	8	9	10

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
306	208	9	33	55	859	271	179	984	93

LSD를 이용한 기수 정렬의 예 (2)

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
306	208	9	33	55	859	271	179	984	93

C[0]	C[1]	C[2]	C[3]	C[4]	C[5]	C[6]	C[7]	C[8]	C[9]
4	1	2	1					1	1
4	5	7	8	8	8	8	8	9	10

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
9	33	55	93	179	208	271	306	859	984

구현

```
public class Radix {
    public static void sort(int[] A) {
        int i, m = A[0], exp = 1, n = A.length;
        int[] B = new int[n];           // 각 기수의 정렬 결과를 임시로 저장할 배열

        for (i = 1; i < n; i++)
            if (A[i] > m)    m = A[i];  // 가장 큰 수를 검색: 이 수에 따라 기수의 수 d가 결정

        while (m / exp > 0) {           // exp가 1이므로, 1의 자리부터 시작 → LSD
            int[] C = new int[10];      // 버킷 할당
            for (i = 0; i < n; i++)     C[(A[i] / exp) % 10]++;       // 낮은 자리부터: LSD
            for (i = 1; i < 10; i++)    C[i] += C[i – 1];
            for (i = n – 1; i >= 0; i--) B[--C[(A[i] / exp) % 10]] = A[i];
            for (i = 0; i < n; i++)
                A[i] = B[i];
            exp *= 10;                  // 다음 높은 자리로 이동
        }
    }
    …
```

시간 복잡성 = $O(d*(n+r))$

4. 병합 정렬(Merge Sort)

- 기본 개념
  - 정렬된 두 개의 서브 리스트를 병합
  - Mergesort overview

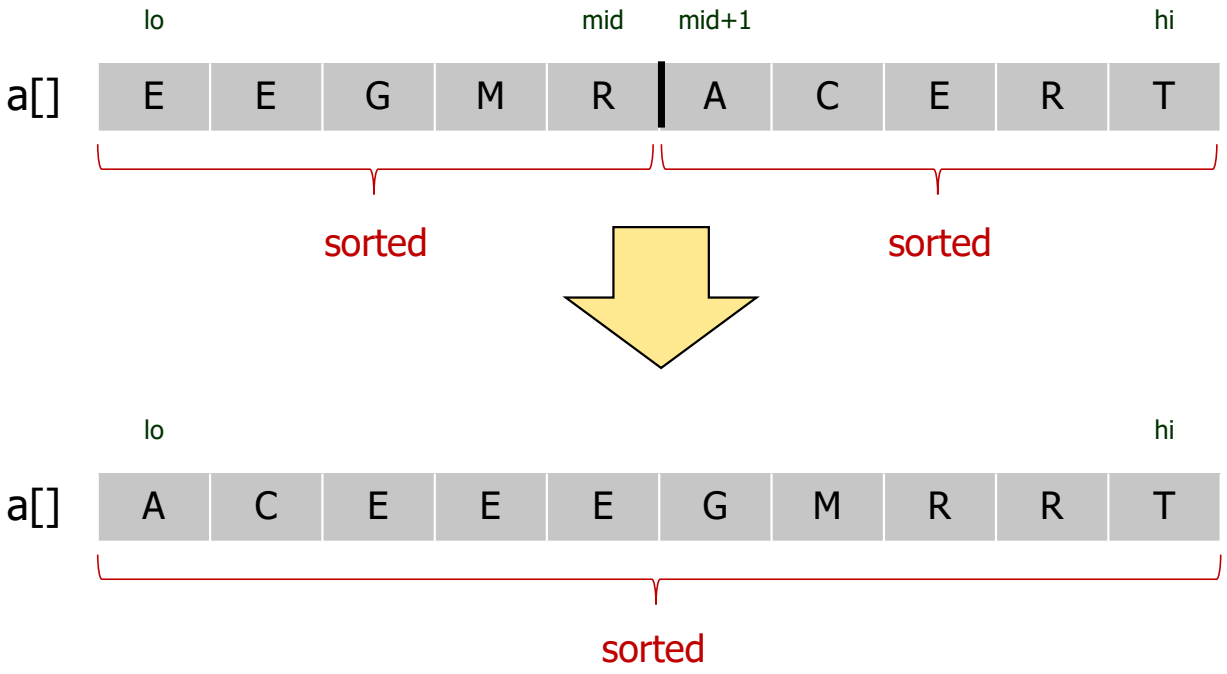
input	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
sort left half	E	E	G	M	O	R	R	S	T	E	X	A	M	P	L	E
sort right half	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
merge results	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

- 병합의 방법
  - 별도의 배열 이용
  - 제자리 병합(In-place merge)

- 정렬의 방법
  - Top-down Mergesort
  - Bottom-up Mergesort

4.1 제자리 병합

- 목표 – 두 개의 정렬된 배열 부분 (a[lo]..a[mid], a[mid+1]..a[hi])를 a[lo]..a[hi]까지 하나로 병합

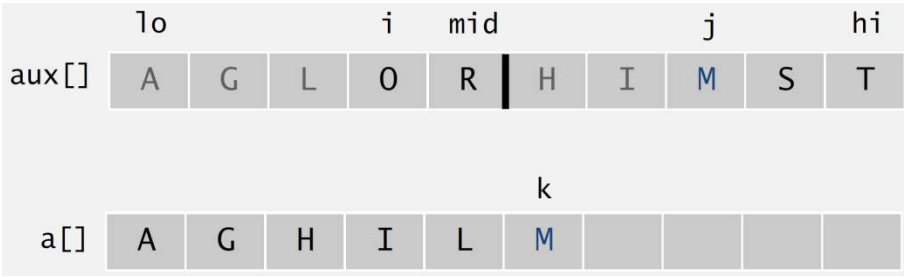


구현

```
private static void merge(Comparable[] a, Comparable[] aux, int lo, int mid, int hi) {
// a[lo .. mid] and a[mid+1 .. hi] 는 이미 정렬

    for (int k = lo; k <= hi; k++)
        aux[k] = a[k];      // aux[] 배열에 a[]의 내용을 일단 복사

    // aux[] 배열을 비교하여 병합된 결과를 a[] 배열에 다시 저장
    int i = lo, j = mid+1;
    for (int k = lo; k <= hi; k++) {
        if (i > mid)                    a[k] = aux[j++];
        else if (j > hi)                a[k] = aux[i++];
        else if (less(aux[j], aux[i]))  a[k] = aux[j++];
        else                            a[k] = aux[i++];
    }
}
```



4.2 Top-Down Merge Sort

- 기본 정책
  - 배열을 두 부분으로 분할
  - 각 부분을 <u>재귀적으로</u> 정렬
  - 두 부분을 병합

input	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
sort left half	E	E	G	M	O	R	R	S	T	E	X	A	M	P	L	E
sort right half	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
merge results	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

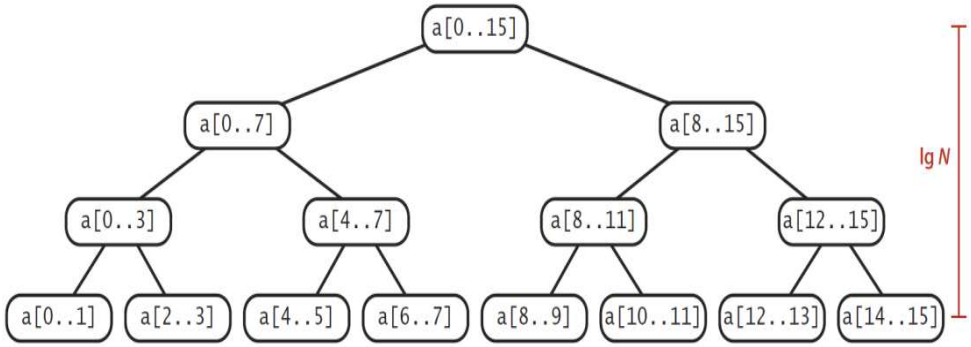
구현

```
public class MergeTD extends AbstractSort {
    private static void merge(. . .)    { /* 4.1절 참조 */ }

    public static void sort(Comparable[] a) {
        Comparable[] aux = new Comparable[a.length];
        sort(a, aux, 0, a.length-1);
    }

    private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi) {
        if (hi <= lo) return;
        int mid = lo + (hi - lo) / 2;
        sort(a, aux, lo, mid);          // 아랫부분 정렬
        sort(a, aux, mid + 1, hi);      // 윗부분 정렬
        merge(a, aux, lo, mid, hi);     // 두 부분을 병합
    }

    . . .
}
```



동작 과정의 예

	a[]															
lo hi	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 0, 0, 1)	E	M	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 2, 2, 3)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 0, 1, 3)	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 4, 4, 5)	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 6, 6, 7)	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 4, 5, 7)	E	G	M	R	E	O	R	S	T	E	X	A	M	P	L	E
merge(a, aux, 0, 3, 7)	E	E	G	M	O	R	R	S	T	E	X	A	M	P	L	E
merge(a, aux, 8, 8, 9)	E	E	G	M	O	R	R	S	E	T	X	A	M	P	L	E
merge(a, aux, 10, 10, 11)	E	E	G	M	O	R	R	S	E	T	A	X	M	P	L	E
merge(a, aux, 8, 9, 11)	E	E	G	M	O	R	R	S	A	E	T	X	M	P	L	E
merge(a, aux, 12, 12, 13)	E	E	G	M	O	R	R	S	A	E	T	X	M	P	L	E
merge(a, aux, 14, 14, 15)	E	E	G	M	O	R	R	S	A	E	T	X	M	P	E	L
merge(a, aux, 12, 13, 15)	E	E	G	M	O	R	R	S	A	E	T	X	E	L	M	P
merge(a, aux, 8, 11, 15)	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
merge(a, aux, 0, 7, 15)	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

성능

- Animation
  - http://www.sorting-algorithms.com/merge-sort

- 길이가 N인 배열의 경우, 비교의 횟수: C(N)
  - $C(N) \le C(\lceil N/2 \rceil) + C(\lfloor N/2 \rfloor) + N$, $C(1) = 0$
  - $C(N) = O(N*\log_2 N)$

- 공간 복잡도: O(N)
  - aux[]을 N/2만큼만 사용하고도 병합 가능?

성능 개선 방법(1)

- 부분의 크기가 특정값보다 작을 경우, 삽입 정렬 사용
  - recursion의 부담을 완화
  - Collections.sort()에서 cutoff = 7로 설정

```
private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi) {
  if (hi <= lo + CUTOFF - 1)  {
     Insertion.sort(a, lo, hi);
     return;
  }
   int mid = lo + (hi - lo) / 2;
   sort(a, aux, lo, mid);
   sort(a, aux, mid + 1, hi);
   merge(a, aux, lo, mid, hi);
}
```

성능 개선 방법(2)

- 이미 정렬되어 있는 경우에는 병합하지 말자!
  - 앞 부분의 제일 큰 원소 <= 뒤 부분의 제일 작은 원소

A	B	C	D	E	F	G	H	I	J	M	N	O	P	Q	R	S	T	U	V
A	B	C	D	E	F	G	H	I	J	M	N	O	P	Q	R	S	T	U	V

```
private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi) {
    if (hi <= lo)
        return;
    int mid = lo + (hi - lo) / 2;
    sort(a, aux, lo, mid);
    sort(a, aux, mid + 1, hi);
    if (less(a[mid], a[mid+1]))
        return;
    merge(a, aux, lo, mid, hi);
}
```

성능 개선 방법(3)

- 병합할 때 a[]를 aux[]에 복사하는 시간을 없애자!
  - a[]와 aux[]의 역할을 교대로 담당

```
private static void merge(Comparable[] aux, Comparable[] a, int lo, int mid, int hi) {
    int i = lo, j = mid+1;
    for (int k = lo; k <= hi; k++) {
        if (i > mid)                    a[k] = aux[j++];     // aux를 a[]에 병합
        else if (j > hi)                a[k] = aux[i++];
        else if (less(aux[j], aux[i]))  a[k] = aux[j++];
        else                            a[k] = aux[i++];
}  }

private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi) {
        if (hi <= lo)          return;
        int mid = lo + (hi - lo) / 2;
        sort(aux, a, lo, mid);              // aux[]와 a[]의 역할을 switch
        sort(aux, a, mid + 1, hi);
        merge(aux, a, lo, mid, hi);
    }
```

함수 호출 전에 a[]를 aux[]에 한번만 복사

4.3 Bottom-Up Merge Sort

- 기본 정책
  - 배열에서 크기 1인 부분(단위 원소)부터 병합
  - 부분의 크기를 2, 4, 8, .. 로 증가하면서 반복

	a[i]															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
sz = 1																
merge(a, aux, 0, 0, 1)	E	M	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 2, 2, 3)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 4, 4, 5)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 6, 6, 7)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 8, 8, 9)	E	M	G	R	E	S	O	R	E	T	X	A	M	P	L	E
merge(a, aux, 10, 10, 11)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	L	E
merge(a, aux, 12, 12, 13)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	L	E
merge(a, aux, 14, 14, 15)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	E	L
sz = 2																
merge(a, aux, 0, 1, 3)	E	G	M	R	E	S	O	R	E	T	A	X	M	P	E	L
merge(a, aux, 4, 5, 7)	E	G	M	R	E	O	R	S	E	T	A	X	M	P	E	L
merge(a, aux, 8, 9, 11)	E	G	M	R	E	O	R	S	A	E	T	X	M	P	E	L
merge(a, aux, 12, 13, 15)	E	G	M	R	E	O	R	S	A	E	T	X	E	L	M	P
sz = 4																
merge(a, aux, 0, 3, 7)	E	E	G	M	O	R	R	S	A	E	T	X	E	L	M	P
merge(a, aux, 8, 11, 15)	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
sz = 8																
merge(a, aux, 0, 7, 15)	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

알고리즘

```
public class MergeBU extends AbstractSort {
    private static void merge(Comparable[] in, Comparable[] out, int lo, int mid, int hi) {
        int i = lo, j = mid+1;
        for (int k = lo; k <= hi; k++) {
            if (i > mid)                    out[k] = in[j++];     // in을 병합하여 out에 저장
            else if (j > hi)                out[k] = in[i++];
            else if (less(in[j], in[i]))    out[k] = in[j++];
            else                            out[k] = in[i++];
    }  }

    public static void sort(Comparable[] a) {
        Comparable[] src = a, dst = new Comparable[a.length], tmp;
        int N = a.length;
        for (int n = 1; n < N; n *= 2) {       // n: 병합할 부분 배열의 크기
            for (int i = 0; i < N; i += 2*n)    // i: 병합할 부분 배열의 위치
                merge(src, dst, i, i+n-1, Math.min(i+2*n-1, N-1)); // (4,4,3)이 남았을 때?
            tmp = src; src = dst; dst = tmp;
        }
        if (src != a)     System.arraycopy(src, 0, a, 0, N);
    }
```

5. 외부 정렬(External Sort)

- 파일처리를 위한 정렬의 필요성
  - Order-by나 group-by, join 등의 효율적인 처리
  - Efficient sequential update
    - new master = old master + transaction

- 내부 정렬 알고리즘들의 문제점
  - 정렬할 데이터들이 메모리에 저장된다고 가정
  - 파일에 저장된 레코드의 수가 많을 경우
    - 모든 레코드들을 메모리에 저장할 수는 없음
    - 배열에 대한 액세스 → 디스크에 대한 액세스?

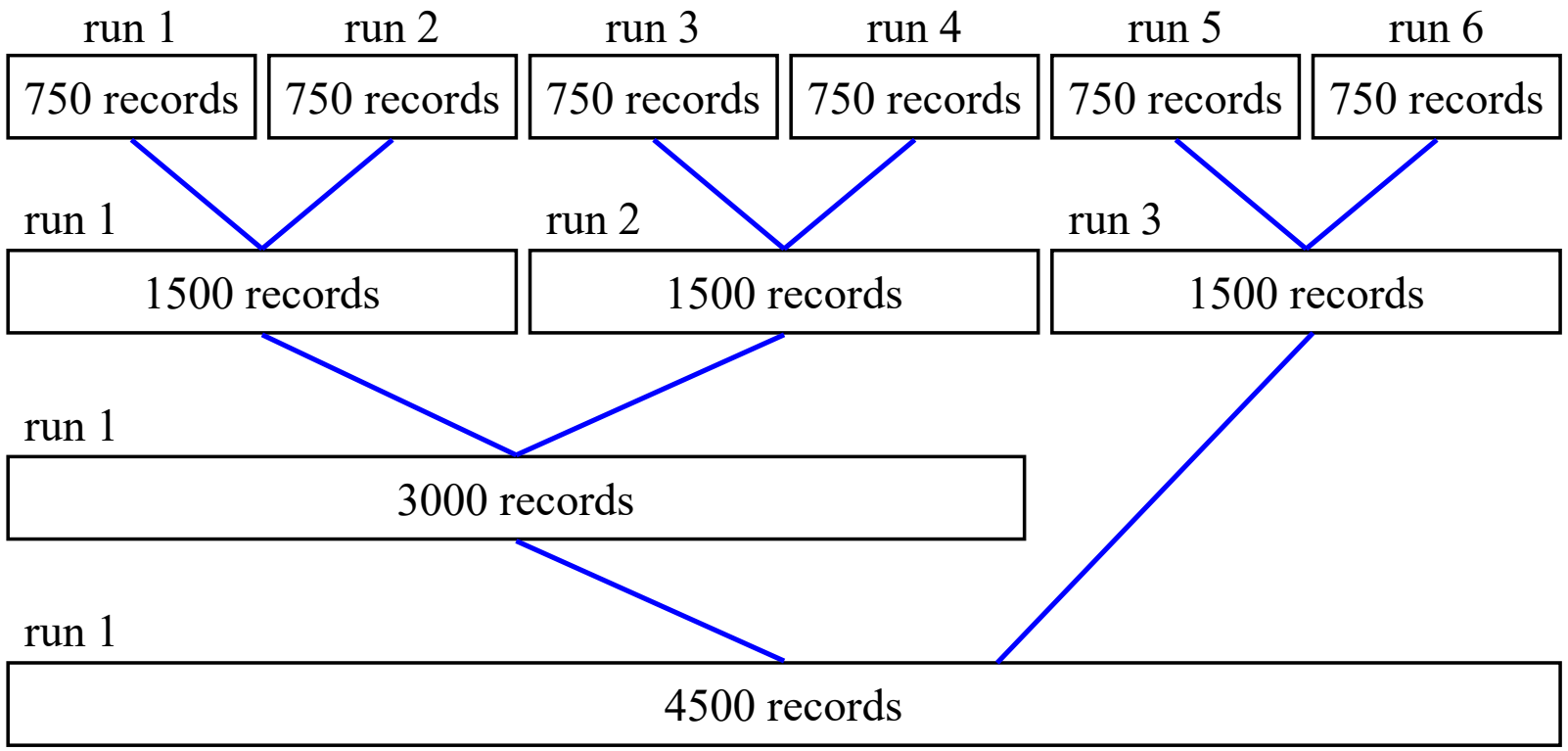
    - Solution: 외부 정렬

5.1 External Sort/Merge Algorithm

- 기본 개념
  - Sorting할 파일을 몇 개의 그룹(run)으로 분할
    - **Run**: 기억장치에 적재 가능
  - 각 run에 대해 내부 정렬 후 다른 파일에 저장
  - 파일에 저장된 run들을 병합하여 다시 파일에 저장
  - 병합된 run의 수가 1이 될 때까지 병합 반복

- 병합 방법에 따른 sort/merge algorithm의 분류
  - Binary Sort/Merge
  - Balanced Binary Sort/Merge
  - Balanced K-way Sort/Merge
  - Polyphase Sort/Merge

외부 정렬의 기본 개념


run 1
750 records
run 2
750 records
run 3
750 records
run 4
750 records
run 5
750 records
run 6
750 records
run 1
1500 records
run 2
1500 records
run 3
1500 records
run 1
3000 records
run 1
4500 records

Binary Sort/Merge

- Sorting Phase
  - 초기 run들을 sorting한 후, 2개의 외부 파일에 저장
- Merging Phase
  - 각 파일에서 하나씩 run을 선택하여 큰 run으로 병합하여 세 번째 파일에 저장
  - 결과 파일을 다시 2 개의 파일로 분배한 후 병합 진행

Binary Sort/Merge의 예

- Input File (Run = 3)
  - 50 110 95 10 100 36 153 40 120 60 70 130 22 140 80

- Sorting Phase의 완료 후
  - File 1: (50 95 110) (40 120 153) (22 80 140)
  - File 2: (10 36 100) (60 70 130)

- 첫 번째 병합의 결과
  - File 3: (10 36 50 95 100 110) (40 60 70 120 130 153) (22 80 140)

- 첫 번째 병합 후 재분배의 결과
  - File 1: (10 36 50 95 100 110) (22 80 140)
  - File 2: (40 60 70 120 130 153)
  - File 3: 비었음

Binary Sort/Merge의 예 - 계속

- 두 번째 병합의 결과
  - File 3: (10 40 36 50 60 70 95 100 110 120 130 153) (22 80 140)

- 두 번째 병합 후 재분배의 결과
  - File 1: (10 40 36 50 60 70 95 100 110 120 130 153)
  - File 2: (22 80 140)
  - File 3: 비었음

- 세 번째 병합의 결과
  - File 3: (10 22 40 36 50 60 70 80 95 100 110 120 130 140 153)

다른 Sort/Merge Algorithm들

- Balanced Binary Sort/Merge
  - 입력 파일 수 = 출력 파일 수 = 2
  - 파일 수가 동일하므로, 파일의 재분배 필요 없음

- Balanced *k*-way Sort/Merge
  - *k*-way Sort/Merge의 Balanced version
  - 파일 수가 증가하므로 더 적은 수의 병합과 run 가짐
  - *k*의 값이 무한히 증가할 수 있는가?

- Polyphase Sort/Merge
  - 입력 파일 수 ≠ 출력 파일 수일 경우에도, 재분배 필요 없음

Ex: Balanced Binary Sort/Merge

- Input File: 50 110 95 10 100 36 153 40 120 60 70 130 22 140 80

- Sorting Phase의 완료 후
  - File 1: (50 95 110) (40 120 153) (22 80 140)
  - File 2: (10 36 100) (60 70 130)

- 첫 번째 병합의 결과
  - File 3: (10 36 50 95 100 110) (22 80 140)
  - File 4: (40 60 70 120 130 153)

- 두 번째 병합의 결과
  - File 1: (10 36 40 50 60 70 95 100 110 120 130 153)
  - File 2: (22 80 140)

- 세 번째 병합의 결과
  - File 3: (10 22 36 40 50 60 70 80 95 100 110 120 130 140 153)

Ex: Balanced *k*-way Sort/Merge

- Input File: 50 110 95 10 100 36 153 40 120 60 70 130 22 140 80

- Sorting Phase의 완료 후
  - File 1: (50 95 110) (60 70 130)
  - File 2: (10 36 100) (22 80 140)
  - File 3: (40 120 153)

- 첫 번째 병합의 결과
  - File 4: (10 36 40 50 95 100 110 120 153)
  - File 5: (22 60 70 80 130 140)
  - File 6:

- 두 번째 병합의 결과
  - File 1: (10 22 40 36 50 60 70 80 95 100 110 120 130 140 153)

Ex: Polyphase Sort/Merge

- Sorting Phase의 완료 후
  - File 1: (50 95 110) (40 120 153) (22 80 140)
  - File 2: (10 36 100) (60 70 130)

- 첫 번째 병합의 결과
  - File 1: (22 80 140)
  - File 2: 비었음
  - File 3: (10 36 50 95 100 110) (40 60 70 120 130 153)

- 두 번째 병합의 결과
  - File 1: 비었음
  - File 2: (10 22 36 50 80 95 100 110 140)
  - File 3: (40 60 70 120 130 153)

- 세 번째 병합의 결과
  - File 1: (10 22 40 36 50 60 70 80 95 100 110 120 130 140 153)

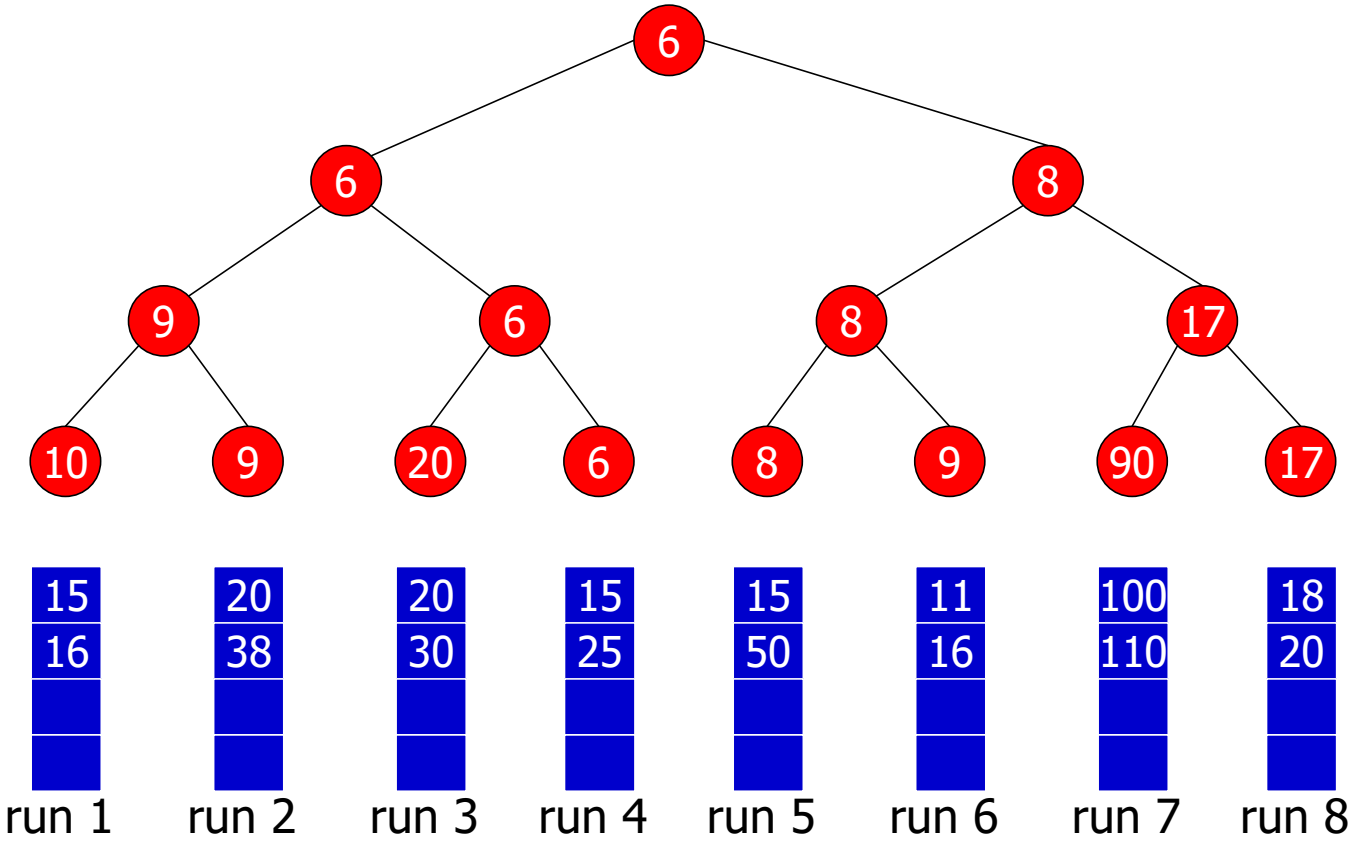
추가적인 고려 사항

- 초기 run의 수가 R인 Binary Sort/Merge
  - Level의 수 = $\lceil \log_2 R \rceil + 1$
  - 전체 병합 단계의 수 = $\lceil \log_2 R \rceil$

- 초기 run의 수가 R인 *k*-way Sort/Merge
  - 전체 병합 단계의 수 = $\lceil \log_k R \rceil$
  - *k*개의 run에서 가장 작은 key 값을 갖는 run 검색
    - Linear search: 전체 비교 수 = $n * (k - 1) * \lceil \log_k R \rceil$
    - Selection tree: 전체 비교 수 = $n * \log_2 k * \lceil \log_k R \rceil = n * \lceil \log_2 R \rceil$

    - Selection Tree? ⇒ See Section 5.8 of HSF

Selection Tree for $k = 8$



5.2 2 Phase Multiway Merge/Sort

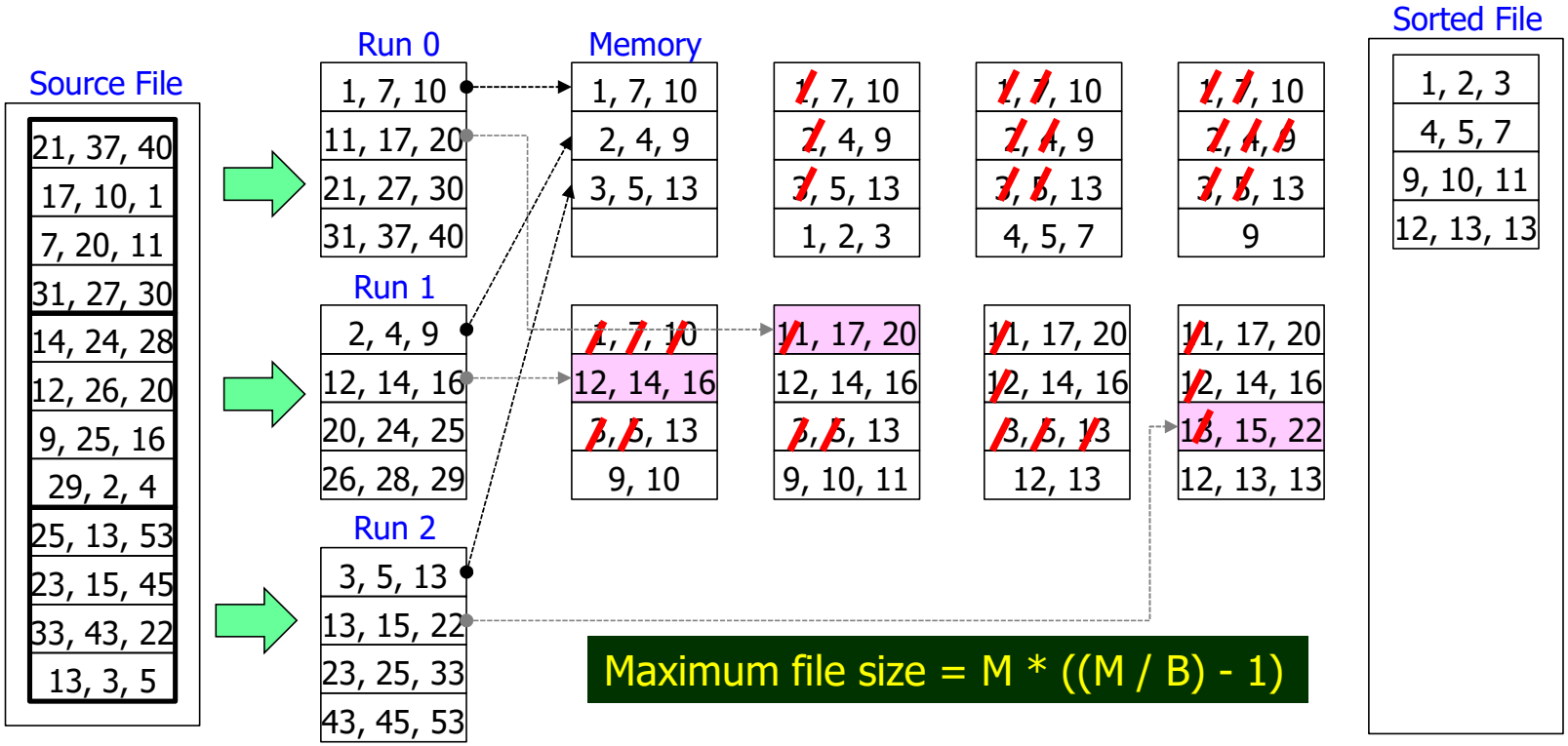
- 2PMM의 기본 개념
  - 2 Phase
    - Sorting Phase + 1번의 Merging Phase
    - 각 phase마다 데이터에 대한 read/write가 1회 수행
  - Multiway
    - 사용할 수 있는 파일의 수는 무제한
  - 2PMM으로 sorting할 수 있는 파일의 크기는 Memory 크기 M에 의해 결정

2PMM Algorithm

- Phase 1 – 정렬 단계
  - 메모리 크기만큼 레코드를 저장
  - 내부 정렬 알고리즘을 이용하여 정렬
  - 정렬된 레코드들을 run에 기록
  - 모든 레코드들이 run에 저장될 때까지 반복

- Phase 2 – 병합 단계
  - 각 run에서 한 블록씩 판독하여 메모리에 저장
  - 가장 작은 키 값을 갖는 레코드를 판단하여, 출력 블록에 저장
  - 출력 블록이 가득 찰 경우, 출력 블록을 디스크에 기록
  - 입력 블록에 레코드가 존재하지 않을 경우, 해당 run에서 다음 블록을 판독하여 메모리에 저장

2PMM의 동작 과정



정렬 단계

병합 단계

Discussion of 2PMM

- Analysis
  - 블록들이 디스크의 임의의 곳에 저장되었다고 가정
    - 블록 액세스 시간 = 15ms
  - 파일은 250,000개의 블록으로 구성
    - 2PMM의 단계별로 모든 블록에 대해 한번의 read와 write 발생
  - 1,000,000 disk I/O's * 15 ms = 15,000 sec = 4+hours.
  - 시간을 줄이는 방법?

5.3 성능 개선 전략

- *k*-way Sort/Merge에서 Parallel I/O
  - 기본 개념
    - Buffer 수: $2k + 2$ (double buffering for I/O)
    - 입력, 출력, 병합 연산을 병렬로 처리
  - Fixed Buffer Allocation
    - 입력 파일마다 2개의 버퍼를 고정 할당
    - 입력 대기 현상 발생 가능
  - Dynamic Buffer Allocation
    - Run들의 최대 키 값을 비교 (Selection Tree 이용)
    - Algorithm: HSF Section 7.10.3 참조

  - 2PMM에서 Parallel I/O 지원 방법?
    - Clustering, Double Buffering
    - Data Replication with Multiple Disks

Run Generation

- 기본 개념
  - Memory 크기 이상으로 초기 run의 크기를 설정
  - Merge pass의 수 감소
- Algorithm
  - Double buffering을 위하여 2개의 I/O 버퍼 이용
  - 키 값을 차례로 입력받아 selection tree 구성
  - Selection tree: 노드 수 = (M – 4) * rec_per_page
  - Tree가 full인 경우, 루트 레코드 출력
  - 출력된 레코드보다 작은 키 값을 갖는 레코드가 입력될 경우, run number를 증가
  - 비교 대상: concatenate(run number, key value)

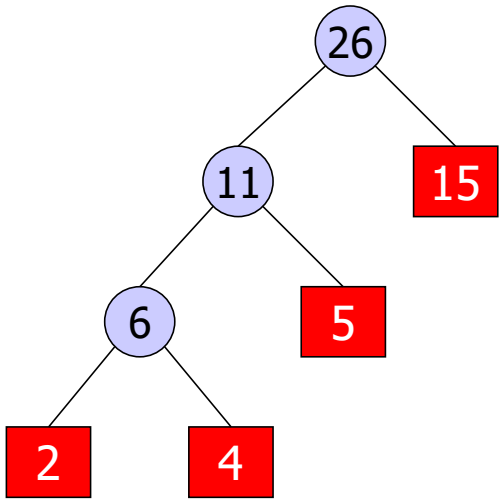
  - HSF Section 7.10.4 참조

Example

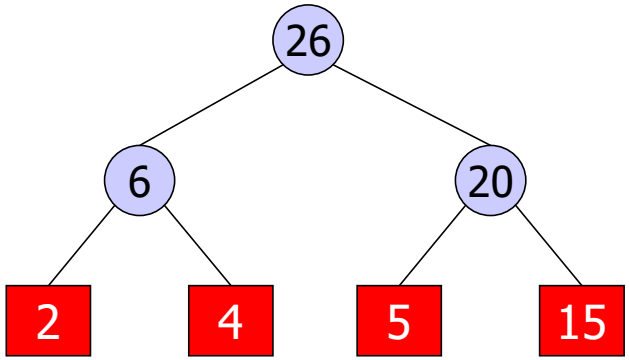
- Memory 크기 = 3
- Input File= (8, 6, 1, 3, 2, 7, 5, 4, 9)
  - 1st run = (1, 3, 6, 7, 8)
    - 8, 6, 1, 3, 2, 7, 5, 4, 9 → 1
    - 8, 6, ~~1~~, 3, 2, 7, 5, 4, 9 → 3
    - 8, 6, ~~1~~, ~~3~~, 2, 7, 5, 4, 9 → 6
    - 8, ~~6~~, ~~1~~, ~~3~~, 2, 7, 5, 4, 9 → 7
    - 8, ~~6~~, ~~1~~, ~~3~~, 2, ~~7~~, 5, 4, 9 → 8
    - ~~8~~, ~~6~~, ~~1~~, ~~3~~, 2, ~~7~~, 5, 4, 9 → done
  - 2nd run = (2, 4, 5, 9)

Optimization - 계속

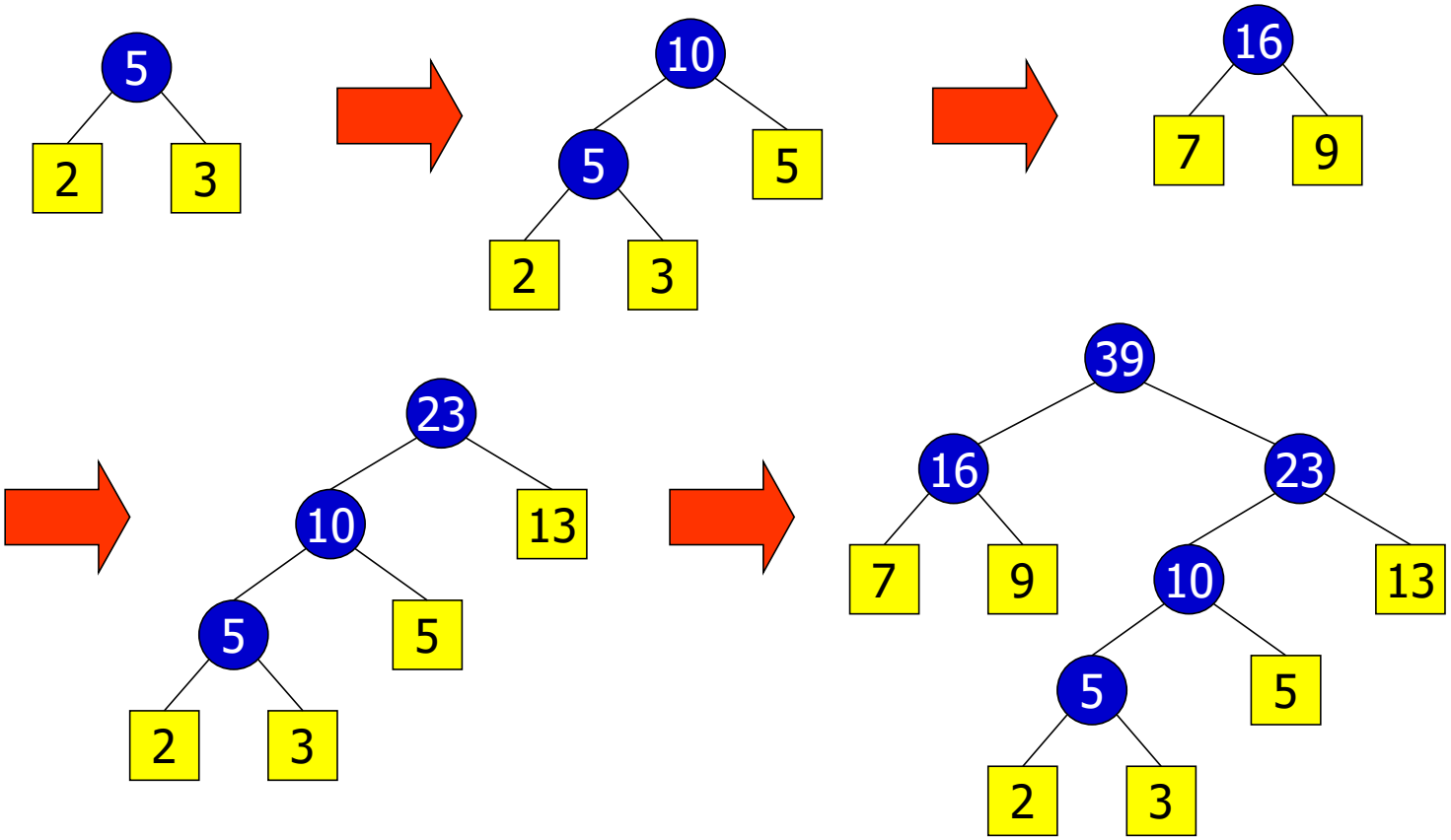
- Optimal Merging of Runs
  - Run generation에 의해 생성된 run의 크기는 상이
  - 각 run의 병합 순서에 따라 실행 속도 차이 발생
  - External path length를 이용하여 비용 계산



Path Length = 43



Path Length = 52

Construction of a Huffman Tree

- Run: 2, 3, 5, 7, 9, 13